a great choice to illustrate both the method itself and why you need to be careful choosing its parameters.

There are also more advanced methods to improve the quality of the hash function, so that it gets closer and closer to uniform distribution.

C.2.3 Conflicts Resolution in Hashing

No matter how good or uniform a hash function we can create, the number of keys, `m`, can grow till it becomes larger than the size of the table, `n`. At that point the *pigeonhole principle* kicks in to rain on our parade.

The *pigeonhole principle* states that if the number of keys to store is larger than the available slots, at some point, we are bound to have two different keys mapped to the same slot. What happens if we try to add both keys to the table? In that case, we have a conflict. We therefore need a way to be able to resolve the conflict and make sure that it is possible to distinguish between the two different keys, and find both of them in a search.

There are two main ways to solve conflicts due to different keys mapping to the same slot:

- *Chaining*: Each element in the array stores a link to another data structure, holding all the keys mapped to that element (see figure C.11). The secondary data structure can be a linked list (usually), a tree, or even another hash table (as in *perfect hashing*, a technique that allows the best possible hashing performance on a *static* set of keys, known in advance). In this case there is no limit to the number of elements that can be stored in the hash table, but performance degrades as we add more and more elements, because (at least some) lists gets longer and longer, and requires more steps to be traversed to find an element.